



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНЖЕНЕРНЫЙ БИЗНЕС И МЕНЕДЖМЕНТ»

КАФЕДРА «ПРОМЫШЛЕННАЯ ЛОГИСТИКА» (ИБМ-3)

Отчет по лабораторной работе № 8
по дисциплине «Технологии машинного обучения»
***«СОЗДАНИЕ ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ ДЕМОНСТРАЦИИ
МОДЕЛЕЙ МАШИННОГО ОБУЧЕНИЯ»***

Студент ИБМ 3-65Б

(Подпись, дата)

Д.А.Росстегаев

Руководитель

(Подпись, дата)

...

ОГЛАВЛЕНИЕ

Описание задания	3
Этапы выполнения работы	4

Описание задания

Разработайте макет веб-приложения, предназначенного для анализа данных.

Вариант 1. Макет должен быть реализован для одной модели машинного обучения. Макет должен позволять:

- задавать гиперпараметры алгоритма,
- производить обучение,
- осуществлять просмотр результатов обучения, в том числе в виде графиков.

Вариант 2. Макет должен быть реализован для нескольких моделей машинного обучения. Макет должен позволять:

- выбирать модели для обучения,
- производить обучение,
- осуществлять просмотр результатов обучения, в том числе в виде графиков.

Для разработки рекомендуется использовать следующие (или аналогичные) фреймворки:

- streamlit
- gradio
- dash

Этапы выполнения работы

На первом этапе был загружен датасет Breast Cancer Wisconsin и выведены первые строки таблицы для просмотра структуры данных. В результате пользователь может просматривать исходные данные непосредственно в веб-приложении.

```
#Загрузка данных
data = load_breast_cancer(as_frame=True)
df = data.frame

#Показать первые строки
st.subheader("Первые строки датасета")
st.dataframe(df.head())
```

Масштабирование и код | Streamlit

localhost:8501 | Streamlit

Настройки

Размер тестовой выборки: 0.40

Выберите модели

Logistic Regression x

Random Forest x

SVM x

Обучить модели

Веб-приложение для демонстрации моделей машинного обучения

Датасет: Breast Cancer Wisconsin Задача: классификация опухоли на доброкачественную или злокачественную

Первые строки датасета

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	n
0	17.99	10.38	122.8	1001	0.1184	0.2776	
1	20.57	17.77	132.9	1326	0.0847	0.0786	
2	19.69	21.25	130	1203	0.1096	0.1599	
3	11.42	20.38	77.58	386.1	0.1425	0.2839	
4	20.29	14.34	135.1	1297	0.1003	0.1328	

Результаты моделей

Модель	Accuracy
--------	----------

Данные были разделены на обучающую и тестовую выборки с использованием метода `train_test_split`.

Размер тестовой выборки задаётся пользователем с помощью ползунка в боковой панели приложения.

```
#Размер тестовой выборки
test_size = st.sidebar.slider(
    "Размер тестовой выборки",
    min_value=0.1,
    max_value=0.5,
    value=0.2,
    step=0.1
)

X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X,
    y,
    test_size=test_size,
    random_state=42,
    stratify=y
)
```

Для моделей Logistic Regression и SVM было выполнено масштабирование признаков с использованием StandardScaler.

Масштабирование позволяет привести признаки к единому диапазону значений и улучшить качество работы моделей.

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

В приложении реализована возможность выбора моделей:

- Logistic Regression;
- Random Forest;
- SVM.

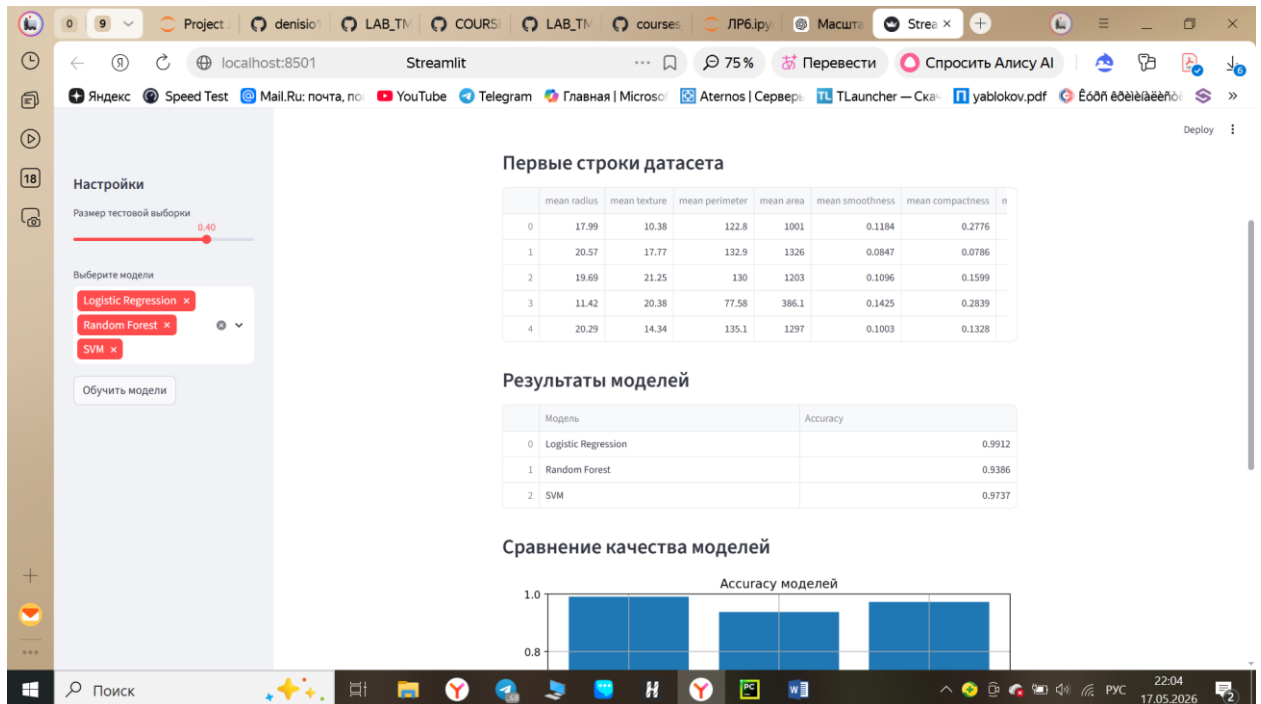
```
#Выбор моделей
selected_models = st.sidebar.multiselect(
    label: "Выберите модели",
    options: ["Logistic Regression", "Random Forest", "SVM"],
    default=["Logistic Regression", "Random Forest", "SVM"]
)
```

После нажатия кнопки «Обучить модели» выполняется обучение выбранных алгоритмов. `model.fit(X_train, y_train)` или `model.fit(X_train_scaled, y_train)` в зависимости от типа модели.

Для оценки качества использовалась метрика Accuracy.

`accuracy = accuracy_score(y_test, y_pred)`

Результаты выводятся в виде таблицы.



Для сравнения качества моделей был построен столбчатый график.

```
#График
st.subheader("Сравнение качества моделей")
fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(
    results_df["Модель"],
    results_df["Accuracy"]
)
ax.set_xlabel("Модель")
ax.set_ylabel("Accuracy")
ax.set_title("Ассурасу моделей")
ax.set_ylim(0, 1)
ax.grid()
st.pyplot(fig)
```

Результат работы

В результате работы было создано веб-приложение для демонстрации моделей машинного обучения.

Приложение позволяет:

- выбирать модели;
- изменять параметры;
- запускать обучение;
- просматривать результаты работы моделей;
- сравнивать качество моделей с помощью графиков.

Наилучший результат показала модель Logistic Regression.

Выводы

В ходе лабораторной работы были изучены основы создания веб-приложений для машинного обучения с использованием Streamlit.

Было реализовано приложение, позволяющее обучать и сравнивать несколько моделей машинного обучения на реальном наборе данных.

Приложение 1. Полный текст кода

```
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

st.title("Веб-приложение для демонстрации моделей машинного обучения")

st.write("""
Датасет: Breast Cancer Wisconsin
Задача: классификация опухоли на доброкачественную или злокачественную
""")

#Загрузка данных
data = load_breast_cancer(as_frame=True)
df = data.frame

#Показать первые строки
st.subheader("Первые строки датасета")
st.dataframe(df.head())

#Разделение признаков и target
```

```

X = df.drop("target", axis=1)
y = df["target"]

#Боковая панель
st.sidebar.header("Настройки")

#Размер тестовой выборки
test_size = st.sidebar.slider(
    "Размер тестовой выборки",
    min_value=0.1,
    max_value=0.5,
    value=0.2,
    step=0.1
)

#Выбор моделей
selected_models = st.sidebar.multiselect(
    "Выберите модели",
    ["Logistic Regression", "Random Forest", "SVM"],
    default=["Logistic Regression", "Random Forest", "SVM"]
)

#Кнопка обучения
if st.sidebar.button("Обучить модели"):

    #train_test_split
    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=test_size,

```

```

    random_state=42,
    stratify=y
)

#Масштабирование
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

#Модели
models = {
    "Logistic Regression": LogisticRegression(max_iter=5000),
    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        random_state=42
    ),
    "SVM": SVC(kernel="rbf")
}
results = []

#Обучение моделей
for model_name in selected_models:
    model = models[model_name]
    if model_name in ["Logistic Regression", "SVM"]:
        model.fit(X_train_scaled, y_train)
        y_pred = model.predict(X_test_scaled)
    else:
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)

```

```

results.append({
    "Модель": model_name,
    "Accuracy": accuracy
})

#Таблица результатов
results_df = pd.DataFrame(results)
st.subheader("Результаты моделей")
st.dataframe(results_df)

#График
st.subheader("Сравнение качества моделей")
fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(
    results_df["Модель"],
    results_df["Accuracy"]
)
ax.set_xlabel("Модель")
ax.set_ylabel("Accuracy")
ax.set_title("Accuracy моделей")
ax.set_ylim(0, 1)
ax.grid()
st.pyplot(fig)

#Лучшая модель
best_model = results_df.sort_values(
    by="Accuracy",
    ascending=False
).iloc[0]
st.success(

```

```
        f"Лучшая модель: {best_model['Модель']} "  
        f"с accuracy = {best_model['Accuracy']:.4f}"  
    )  
else:  
    st.info("Выберите модели и нажмите кнопку 'Обучить модели'")
```